

In the example given below we are dynamically creating list objects by calling a Python function and clicking the button.

The source code is:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>DOM API in PyScript</title>
<link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />

<script defer src="https://pyscript.net/alpha/pyscript.js">

</script>
</head>
<body>
<div id="shopping-list">
<p>Shopping List</p>
<ul>
<li>milk</li>
<li>eggs</li>
<li>bread</li>
</ul>
<input id="new-item" type="text" placeholder="Add new item">
<button>Add</button>
</div>
<py-script>
from pyodide import create_proxy

def on_add_click(event):
input_new_item = document.querySelector("#new-item")

if input_new_item.value:
child = document.createElement("li")
child.innerText = input_new_item.value

parent = document.querySelector("#shopping-list ul")
parent.appendChild(child)

input_new_item.value = ""

button = document.querySelector("button")
button.addEventListener("click", create_proxy(on_add_click))
</py-script>
</body>
</html>
```

Figure 4 shows the output for this example code.

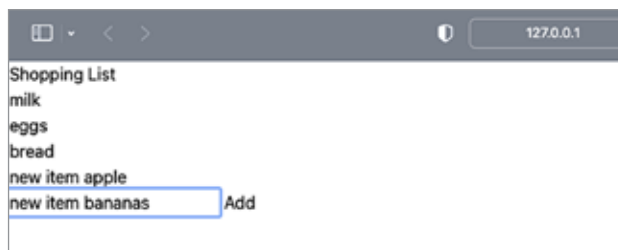


Figure 4: Dynamic list creation

Flexible framework

A flexible framework can be leveraged to create and share new pluggable and extensible components directly in Python. The `<py-repl>` tag creates an REPL component that is rendered to the page as a code editor, allowing you to write executable code inline.

The code given here depicts the `py-repl` tag:

```
<html>
<head>
<link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
<script defer src="https://pyscript.net/alpha/pyscript.js"></script>
</head>
<py-repl></py-repl>
</html>
```

Figure 5 shows the output for the above code.



Figure 5: `py-repl` tag

Visual application development

Here, readily available curated UI components can be used, such as buttons, containers, text boxes, and more. The following tags can be used to add visual attributes to your HTML page.

- `<py-inputbox>`: Adds an input box that can be used to prompt users to enter input values.
- `<py-box>`: Creates a container object that can be used to host one or more visual components that define how elements of `<py-box>` should align and show on the page.
- `<py-button>`: Adds a button to which authors can add labels, and event handlers for actions on the button, such as `on_focus` or `on_click`.
- `<py-title>`: Adds a static text title component that styles the text inside the tag as a page title.